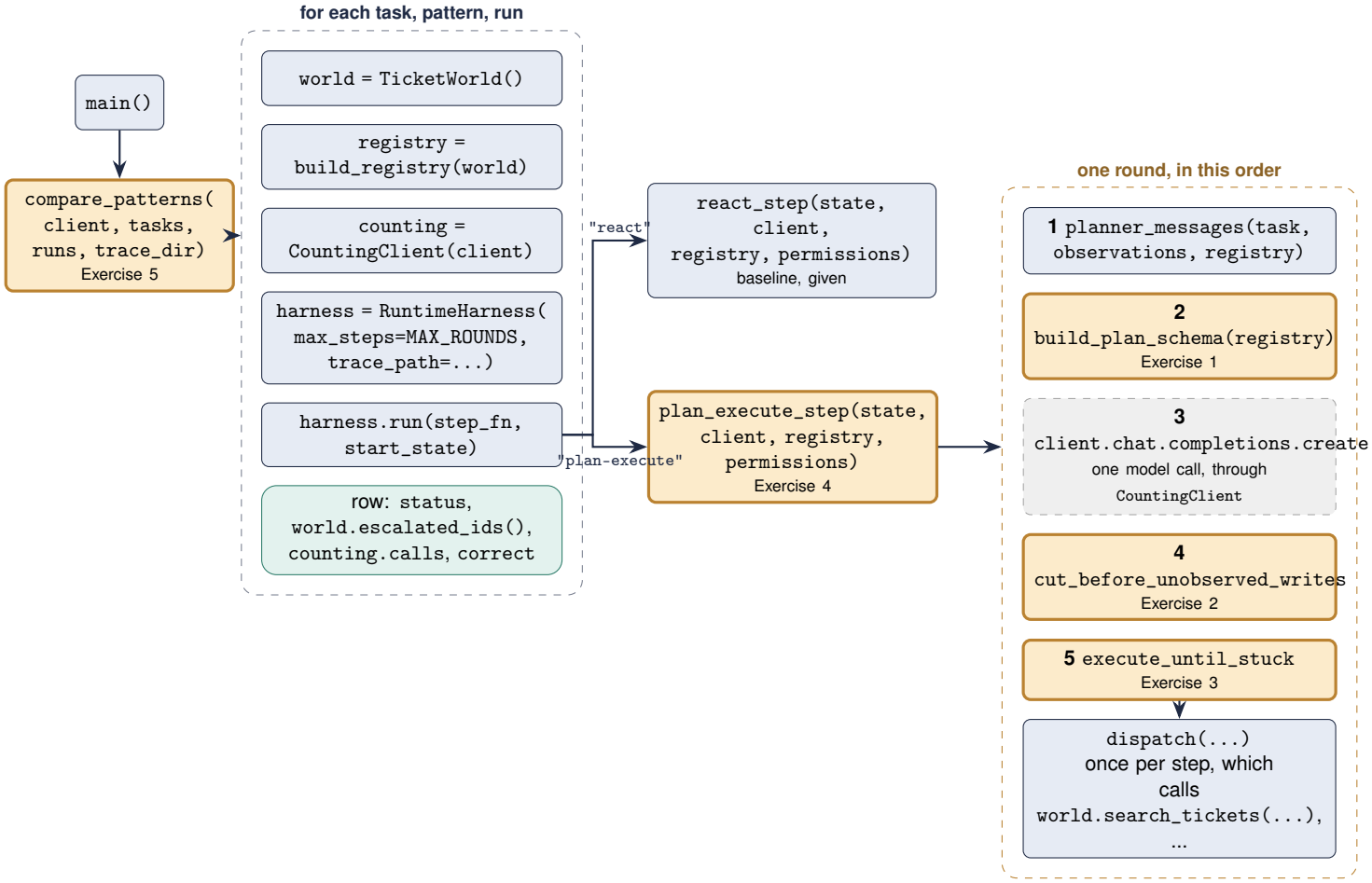
Lab 8a call flow, 1 of 6: who calls whom

Run `python3 scaffold.py` and this is the whole chain, left to right. Gold boxes are your five exercises.



Fresh state every run. *A new `TicketWorld`, a new registry that points at* its *methods, and a new `CountingClient`, so one run never sees another run's escalations, and `.calls` counts only this run.*

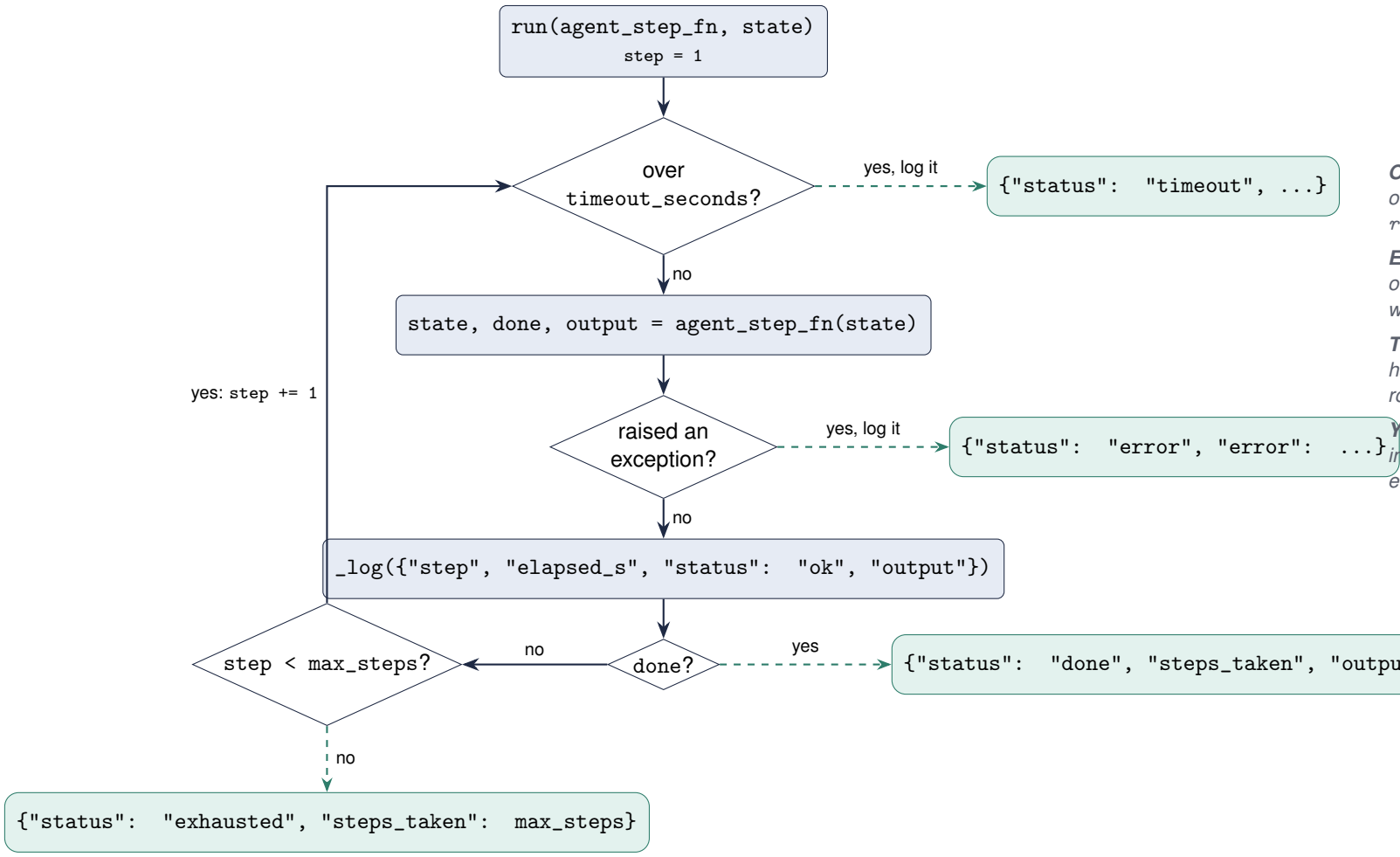
The harness passes only `state`. *It calls `step_fn(state)` and nothing else. The client, registry, and permissions that a step needs have to be tied into `step_fn` before you hand it to `run`.*

Same harness, two step functions. `react_step` *and* `plan_execute_step` *have the same shape: take a state, return* `(new_state, done, output)`. *That is why one harness can run either.*

given code | you write this | outside your code | data passed along | calls | returns

Lab 8a call flow, 2 of 6: inside `RuntimeHarness.run`

Given, unchanged from Week 7. It owns the loop; your step function owns one round.



One pass of this loop = one round *of `plan_execute_step`, or one turn of `react_step`.*

Every round is written to the trace file *as one JSON line. That is what you read afterwards to see what happened.*

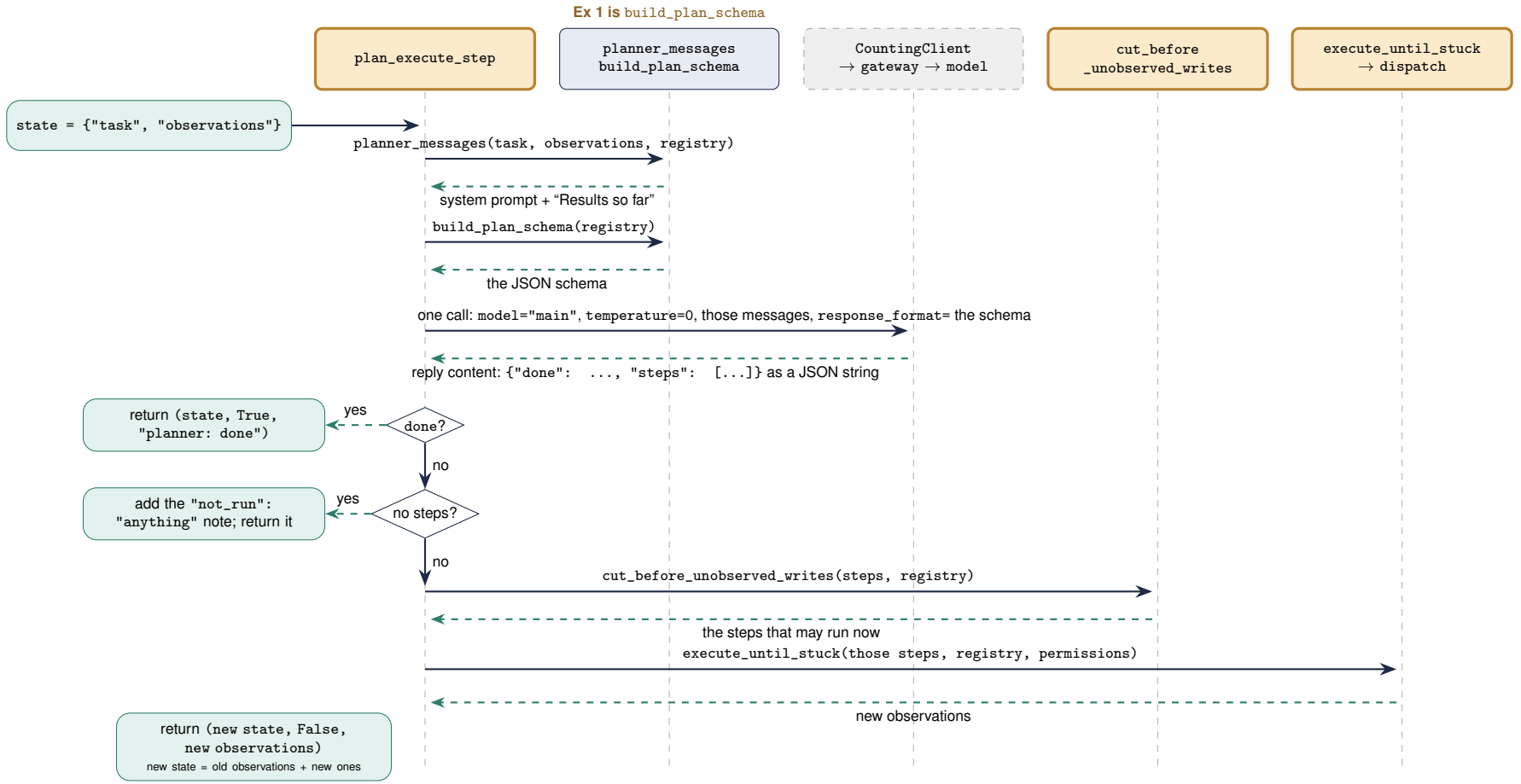
The loop never looks inside *`state`. It just hands back whatever your step returned, next round.*

Your step decides when to stop *by returning `done = True`. If it never does, the loop ends at `max_steps` with `"exhausted"`.*

given code | you write this | outside your code | data passed along | calls | returns

Lab 8a call flow, 3 of 6: one round of `plan_execute_step`

Exercise 4. Read top to bottom: this is the order of calls in a single round, as the README specifies.



Exactly one model call per round. *Everything else in the round is ordinary Python: no model involved.*

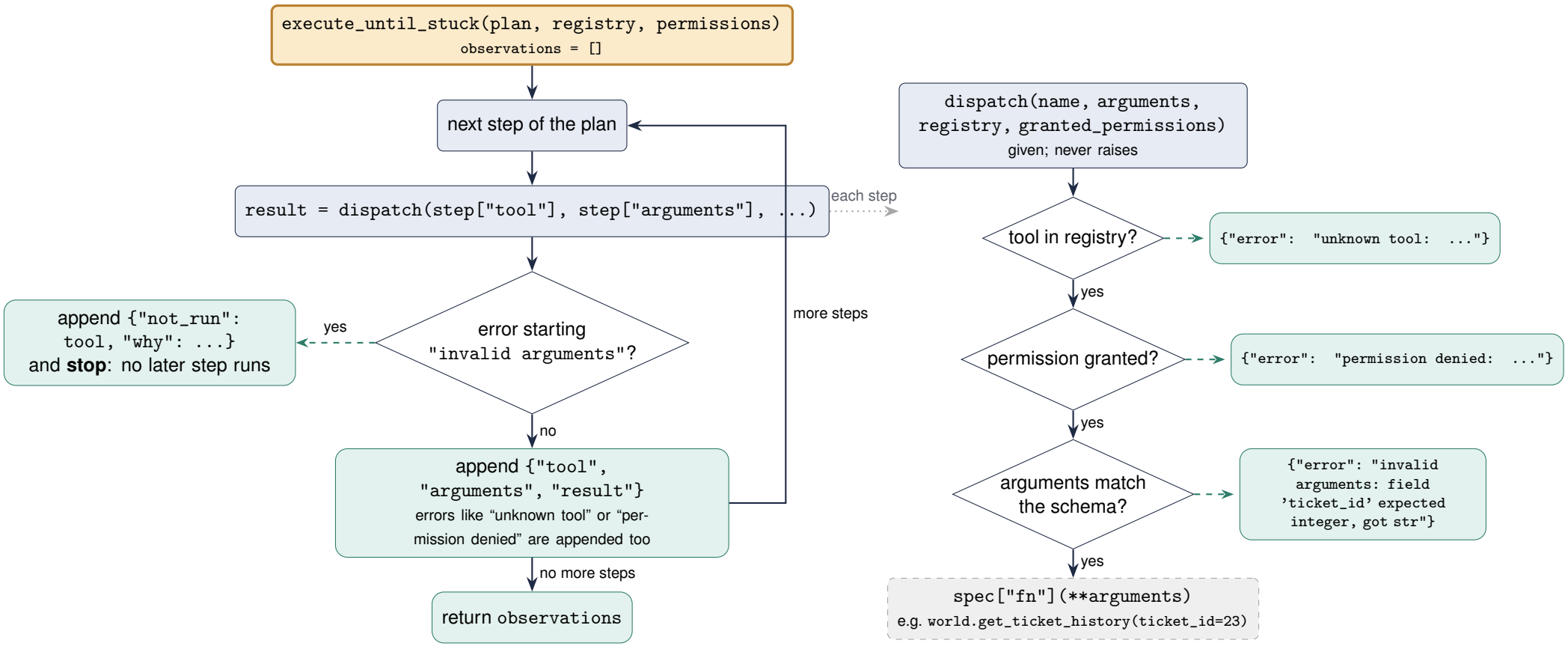
The model plans; your code executes. *The model only ever returns a list of steps. Nothing in that reply runs until `execute_until_stuck` dispatches it.*

Never change the state you were given. *Build a new dict. The harness passes your returned state into the next round.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

Lab 8a call flow, 4 of 6: `execute_until_stuck` and `dispatch`

Exercise 3 on the left calls the given `dispatch` on the right, once per step, in order.



Why stop at "invalid arguments"? *In the lecture, the planner wrote placeholders such as `"{{search_tickets_result[0]}}"` where an id should be. `dispatch` rejects that as the wrong type. Everything after it in the plan was written on the same wrong assumption, so it must not run.*

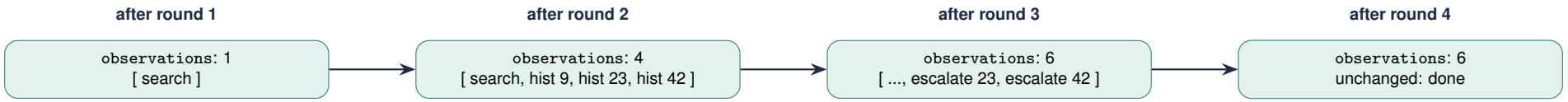
Why record the other errors and carry on? *An unknown tool or a denied permission is a real result the planner should see next round. It is information, not a broken plan.*

given code | you write this | outside your code | data passed along | calls | returns

Lab 8a call flow, 5 of 6: how `state` grows, round by round

A real run of the first task, plan and replan, 29 September. Four rounds, four model calls, correct: tickets 23 and 42.

Round	The planner sees (`observations`)	It plans (`steps`)	After cut and execute
1	*none yet*	`search_tickets("unresolved")`	observation 1: result `[9, 23, 42]`
2	1 observation: the three ids	`get_ticket_history` for 9, 23, 42	observations 2–4: 9 was “escalated to tier 2”; 23 and 42 never were
3	4 observations, histories included	`escalate_ticket` for 23 and 42	observations 5–6: both escalated
4	6 observations	`done = true`, no steps	harness gets `done = True`, returns `"done"`



Why this works when a one-shot plan failed. *The ids 23 and 42 only exist after round 1 runs. Each round, the planner writes only what it can see, so it never needs a placeholder.*

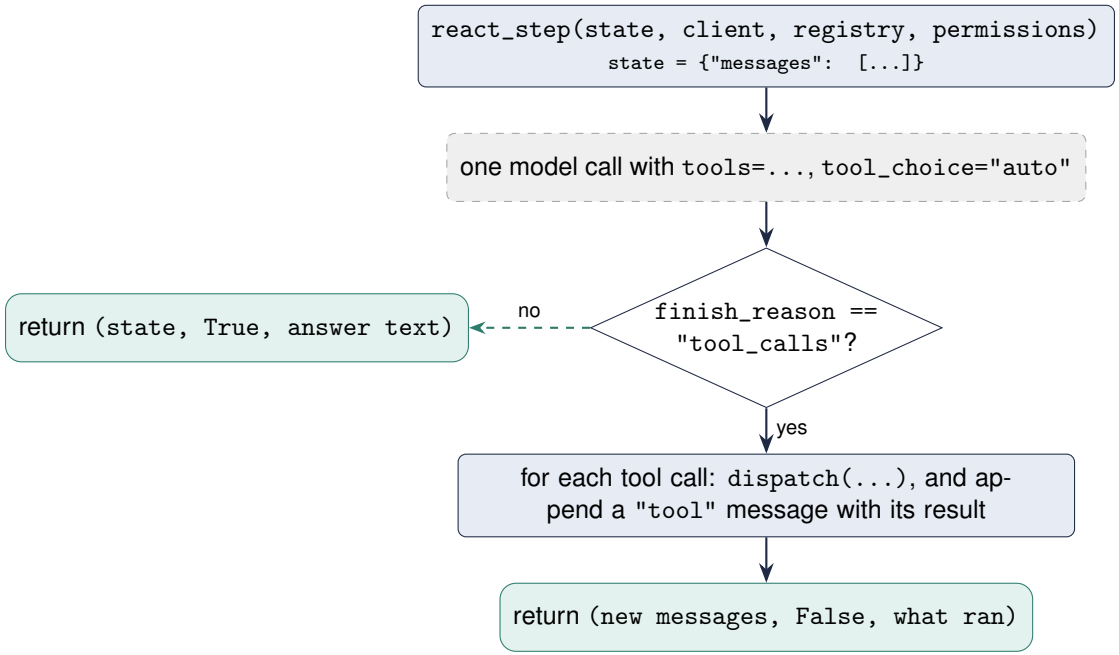
Where Exercise 2 bites. *If a round’s plan reads histories* and *escalates, the escalations are cut and re-planned next round, after the histories are visible.*

Read it yourself. *Each row above is one line of* `trace_escalate-never-escalated_plan-execute_1.jsonl`. *Open it after a run.*

given code | you write this | outside your code | data passed along | calls | returns

Lab 8a call flow, 6 of 6: the baseline, `react_step`

Given. Same harness, same `dispatch`, same tools. The model picks tool calls turn by turn instead of planning.



Turn	What the model asked for (real run, first task, 29 September)	What went wrong
1	`search_tickets("unresolved")` → `[9, 23, 42]`	
2	histories of 9, 23, 42 *and* `escalate_ticket` 9 and 42, all in one turn	escalations chosen before the histories came back
3	`escalate_ticket` 42 again	42 escalated twice
4	final answer: the rest "are either resolved or have already been escalated"	23 was neither; result `[9, 42, 42]`

Messages, not observations. *ReAct's state is the whole conversation. Each turn appends the model's tool calls and one `"tool"` message per result, so the model sees everything at the next call.*

Nothing sits between the model and the tools. *There is no step where your code can cut a write, which is exactly the job of Exercise 2 in the planner.*

given code | you write this | outside your code | data passed along | → calls | - - → returns